optany: Unified Text Optimization can Outperform Specialized Systems

Anonymous Author(s)

Abstract

Can a single LLM-based optimization system match specialized tools across fundamentally different domains? We show that when optimization problems are formulated as improving a text artifact evaluated by a scoring function, a single AI-based optimization system—supporting single-task search, multi-task search with cross-problem transfer, and generalization to unseen inputs—achieves state-of-the-art results across six diverse tasks. Our system discovers agent architectures that nearly triple Gemini Flash's ARC-AGI accuracy (32.5% → 89.5%), finds scheduling algorithms that cut cloud costs by 40%, generates CUDA kernels where 87% match or beat PyTorch, and outperforms AlphaEvolve's reported circle packing solution (n=26). Ablations reveal that structured diagnostic feedback (side information) yields faster convergence and higher final scores than score-only feedback, and that multi-task search can outperform independent optimization given equivalent per-problem budget through cross-task transfer. Together, we show for the first time that text optimization with LLM-based search is a general-purpose problem-solving paradigm, unifying tasks traditionally requiring domain-specific algorithms under a single framework.

CCS Concepts

• **Computing methodologies** → **Natural language processing**; *Neural networks*; **Artificial intelligence**.

Keywords

LLM optimization, text artifact optimization, evolutionary search, prompt engineering, agentic systems, Pareto optimization

ACM Reference Format:

Anonymous Author(s). 2026. optany: Unified Text Optimization can Outperform Specialized Systems. In *Proceedings of ACM Conference on AI and Agentic Systems (ACM CAIS '26)*. ACM, New York, NY, USA, 13 pages. https://doi.org/10.1145/nnnnnnn.nnnnnnn

1 Introduction

Large language models can serve as effective optimizers when paired with automated evaluation. FunSearch [5] evolves Python functions to discover mathematical constructions that surpass known bounds. AlphaEvolve [14] extends the idea to broader code optimization, improving a 56-year-old matrix multiplication bound and designing scheduling heuristics for Google's data centers, but it operates exclusively on code artifacts, in single-task mode (one problem at a time). GEPA [1] achieves state-of-the-art prompt optimization with generalization to unseen inputs, but is limited to prompts; MIPROv2 [15] similarly targets prompt and few-shot selection. Despite strong results within their respective artifact types, no existing system has been applied to agent architectures, numerical optimization, or image generation, and no single system has demonstrated effectiveness across fundamentally different domains simultaneously.

We observe that a wide range of problems can be formulated as optimizing a text artifact. Whether the artifact is a CUDA kernel, a cloud scheduling policy, an agent architecture, Scalable Vector Graphics (SVGs), or a system prompt, the structure is the same: serialize the artifact as a string, evaluate it, and let an LLM propose improvements based on diagnostic feedback. This observation suggests a much simpler interface and a uniform algorithm is possible.

We present optany, a declarative API that implements this insight. The user provides a seed artifact (or, in seedless mode, just a natural-language objective), an evaluator that returns a score and optional diagnostic feedback, and optionally a dataset. The system handles prompt construction, reflection, candidate selection, and search strategy. This declarative design, inspired by DSPy's [8] principle of *programming—not prompting*, means the same API call works whether one is optimizing an LLM prompt, an agent architecture, or an image.

Our contributions are as follows:

(1) **A single LLM-based Text Optimization system matches or surpasses domain-specific tools across six fundamentally different domains.** We are the first to show that a single system (our proposed optany) can optimize code, prompts, agent architectures, numerical configurations, and images, achieving state-of-the-art results in each. Our system discovers agent architectures that nearly triple ARC-AGI accuracy (32.5% → 89.5%), finds scheduling algorithms that cut cloud costs by 40%, generates CUDA kernels where 87% match or beat PyTorch baselines, create custom solver code matching and outperforming Optuna in numerical optimization, and outperforms AlphaEvolve's solution on circle packing. This establishes LLM-based text optimization as a general-purpose problem-solving paradigm, not limited to code or prompts.

(2) **Three optimization modes—single-task, multi-task, and generalization—unified under one interface, including the first multi-task mode.** Existing LLM-evolution systems each support exactly one mode. AlphaEvolve [14], OpenEvolve [19], and ShinkaEvolve [9] operate in single-task mode: optimizing one code artifact for one problem at a time. GEPA [1] and MIPROv2 [15] operate in generalization mode: optimizing a prompt to perform well on unseen inputs, but only for prompts. No prior system supports *multi-task search*, where solving a batch of related problems together enables cross-transfer of discovered optimization patterns. optany unifies all three modes under one interface: multi-task search on CUDA kernels outperforms independent single-task optimization given equivalent per-problem budget (§5.8), and generalization extends beyond prompts to agent architectures (§5.3) and scheduling policies

ACM CAIS '26, San Jose, CA, USA
2026. ACM ISBN 978-x-xxxx-xxxx-x/YYYY/MM
https://doi.org/10.1145/nnnnnnn.nnnnnnn

figures/header_image.png

Figure 1: The `optany` loop: a text artifact x is passed to an evaluator $f(x)$ which returns a score plus diagnostic feedback (SI), which is consumed by an LLM proposer to produce an improved artifact. The same API instantiates across domains: code optimization, prompt tuning, agent architecture search, and policy discovery.

(§5.2). All optimization modes are expressed through the same `optany` API.

(3) **Side information as a first-class evaluator contract.** Prior frameworks support diagnostic feedback through ad-hoc, framework specific mechanisms. `optany` elevates it to a uniform API contract: any diagnostic—stack traces, profiler data, rendered images, structured error reports—flows to the proposer through one interface. Ablations show that side information yields 4-6× faster convergence and a 3.8-point improvement in final test performance versus score-only feedback (§5.9).

We achieve these results by extending the Pareto-based search of Agrawal et al. [1] (originally studied only for prompt optimization) to arbitrary text artifacts, adding single-task and multi-task modes. Candidates are selected based on per-example or per-metric Pareto dominance rather than aggregate scores, preserving complementary strengths across iterations. Table 2 provides a detailed comparison.

We evaluate `optany` across six primary domains spanning all three optimization modes (Table 1), with two additional domains (blackbox mathematical optimization and 3D modeling) in the appendix as preliminary demonstrations. Key results include: (i) evolved agent architectures nearly triple Gemini Flash's ARC-AGI accuracy (32.5% → 89.5%); (ii) discovered cloud scheduling algorithms cut costs by up to 40%; (iii) 87% of generated CUDA kernels match or beat PyTorch baselines from KernelBench, with multi-task mode outperforming dedicated single-task optimization; (iv) prompt optimization improves GPT-4.1-mini's AIME-2025 accuracy from 46.67% to 60.00%; and (v) our circle packing solution outperforms AlphaEvolve's published one. Ablations show that side information yields 4-6× faster convergence and +3.8 test points over score-only feedback.

2 Related Work

LLM-based program evolution. AlphaEvolve [14] pioneered the LLM-evolution paradigm, using Gemini models with island-based MAP-Elites [13] to discover algorithms for Google's infrastructure. OpenEvolve [19] provides an open-source reimplementation with model-agnostic support. ShinkaEvolve [9] extends the paradigm with novelty-based rejection sampling for sample efficiency and adaptive LLM ensemble selection for diversity. FunSearch [5] applies evolutionary LLM search to mathematical discovery. EvoPrompting [3] evolves code for neural architecture search. All operate exclusively in single-task mode and expose framework-specific abstractions (island topologies, prompt samplers, evolve-block markers). `optany` strips the interface to its declarative essence, adds multi-task and generalization modes, and elevates diagnostic feedback to a first-class API concept.

Prompt optimization. GEPA [1] combines reflective mutation with a Pareto-based search technique for prompt optimization, outperforming both MIPROv2 [15] and GRPO [18]. `optany` uses GEPA's evolutionary search algorithm as the optimization backend, extending it beyond prompts to arbitrary text artifacts. Other prompt optimization methods include OPRO [21], APE [24], ProTeGi [17], and PromptBreeder [6]. TextGrad [22] uses LLM-generated "gradients" for text optimization.

LLM self-improvement and reflection. Reflexion [20] uses verbal reinforcement for agent self-correction. Self-Refine [11] applies iterative self-feedback. Evolution through Large Models [10] explores

LLMs as mutation operators. optany's SI mechanism generalizes these ideas by making diagnostic feedback a declarative evaluator contract rather than a hardcoded self-critique.

Agent architecture search. ADAS [7] and AFlow [23] search over agent architectures. optany's generalization mode subsumes these as special cases: the artifact is the agent code, the evaluator runs it on tasks, and the system evolves both architecture and prompts jointly.

3 The optany API

3.1 Core Interface

At its simplest, optany requires a seed artifact and an evaluator. The evaluator takes a candidate string and returns a score (higher is better) alongside an optional Side Information (SI) dictionary containing diagnostic feedback the proposer reads during reflection:

```
import optany as oa

def evaluate(candidate: str) -> tuple[float, dict]:
    result = execute_code(candidate)
    return result.score, {
        "Error": result.stderr,
        "Output": result.stdout,
        "Runtime": f"{result.time_ms:.1f}ms",
    }

result = oa.optany(
    seed_candidate="<your artifact>",
    evaluator=evaluate,
)
```

SI can include open-ended text, structured data, multiple subscores, or images (via oa.Image) for Vision-capable LLMs (VLM).

The full optany signature is:

```
def optany(
    seed_candidate=None, # Starting artifact
    evaluator=..., # fn(candidate) -> Score + SI
    dataset=None, # Training examples
    valset=None, # Validation set
    objective=None, # Natural language goal
    background=None, # Domain knowledge
    config=None, # Engine settings
) -> OptimizationResult:
```

Specifically, optany doesn't require mutation prompts, task-specific templates, island configurations, or EVOLVE-BLOCK markers (all common in prior frameworks). The user declares the *what* (artifact, evaluator, domain knowledge), and optany, through its optimization backends, handles the *execution*.

Seedless mode. In domains where providing even a starting artifact is difficult, or where writing even a bad seed requires domain expertise (e.g., 3D modeling), the user can just provide a natural-language objective as an argument in place of the seed_candidate argument and the LLM bootstraps the first candidate from scratch. Seedless mode makes the system accessible to users who can *specify* what they want but not *implement* it. Appendix C demonstrates it on a 3D modeling task.

3.2 Three Optimization Modes

Which mode is active depends solely on whether dataset and valset are provided:

Single-Task Search. No dataset. The candidate *is* the solution; the evaluator scores it directly. This is the mode that AlphaEvolve and OpenEvolve operate in. Example: in circle packing (§5.6), the artifact is the packing algorithm and the evaluator returns the packing score plus geometric diagnostics.

Table 1: Summary of experimental results across six domains. "Mode" indicates which optimization paradigm is used: S = single-task search, M = multi-task search, G = generalization. All results use optany with the indicated proposer LLM.

Domain	Mode	Proposer	Key Result
Agent Skills (§5.1)	G	Claude Opus 4.6	100% pass rate (+20.7pp)
Cloud Sched. (§5.2)	G	Gemini 3 Pro	40.2% cost savings
ARC-AGI (§5.3)	G	Gemini 3 Flash	89.5% test (+57pp)
AIME Prompts (§5.4)	G	GPT-5	60.0% test (+13.3pp)
CUDA Kernels (§5.5)	M	GPT-5	87% match baseline
Circle Pack. (§5.6)	S	GPT-5	Beats AlphaEvolve
Math Opt. (§B)	S	GPT-5	Wins 7/10 vs Optuna
3D Modeling (§C)	S	Cloud Opus 4.6	Generates a 3D unicorn

Multi-Task Search. A dataset of related tasks is provided; insights from solving one help solve the others. Example: in CUDA kernel generation (§5.5), each task is a PyTorch operation to accelerate. Multi-task mode discovers optimization patterns that transfer across problems, converging faster and solving more problems than single-task runs (§5.8). No prior LLM-evolution framework supports this mode.

Generalization. Both dataset and valset are provided; the optimized artifact must perform well on unseen examples. This is the mode that GEPA's prompt optimization [1] operates in.
optany generalizes the pattern to any text artifact. Example: in agent architecture discovery (§5.3), the artifact is the entire agent, and it must generalize to unseen ARC-AGI puzzles.

4 Method

optany currently extends and manages information atop GEPA [1], an algorithm originally studied primarily in the context of prompt optimization and code search, as the optimization backend. The system overview is shown in Figure 1. We describe the two mechanisms that underpin its effectiveness and contrast optany with prior frameworks.

4.1 Problem Formulation

We formalize the text optimization problem as follows. Let $\mathcal{X}$ denote the space of text artifacts (strings). An evaluator $f : \mathcal{X} \times \mathcal{E} \cup \{\bot\} \to \mathbb{R} \times \mathcal{I}$ maps an artifact $x \in \mathcal{X}$ and an (optional) example $e \in \mathcal{E} \cup \{\bot\}$ to a score $s(x, e) \in \mathbb{R}$ and side information $\iota(x, e) \in \mathcal{I}$, i.e., $f(x, e) = (s(x, e), \iota(x, e))$. The three modes correspond to:

Single-task search: $\mathcal{E} = \emptyset$; maximize $s(x)$ directly. The artifact *is* the solution (e.g., a packing algorithm).

Multi-task search: Given a dataset $\mathcal{D} = \{e_1, \ldots, e_n\}$ of related problems, find an artifact $x \in \mathcal{X}$ (e.g., a kernel-generation prompt) maximizing $\frac{1}{n} \sum_{i=1}^{n} s(x, e_i)$. Cross-transfer arises because the Pareto frontier preserves patterns that work across problems.

Generalization: Given a training set $\mathcal{D}_{\text{train}}$ and a validation set $\mathcal{D}_{\text{val}} = \{e_1^{\text{val}}, \ldots, e_k^{\text{val}}\}$, find an artifact $x \in \mathcal{X}$ maximizing

Table 2: Comparison of `optany` with prior LLM-evolution frameworks. Only `optany` supports multi-task search and generalization modes, and provides diagnostic feedback as a first-class API concept.

Feature	AlphaEv.	OpenEv.	ShinkaEv.	optany
Single-task search	✓	✓	✓	✓
Multi-task search				✓
Generalization				✓
SI as API contract				✓
Image feedback				✓
Declarative interface				✓
No mutation templates				✓
Open-source		✓	✓	✓

$\frac{1}{k}\sum_{j=1}^{k} s\left(x, e_j^{\text{val}}\right)$. Search uses feedback from $\mathcal{D}_{\text{train}}$, while $\mathcal{D}_{\text{val}}$ measures generalization to unseen examples. This generalizes classical machine learning: the artifact may be a prompt, an agent, or a policy.

4.2 Side Information (SI)

Popularly used numerical optimization methods like gradient descent reduce all diagnostic context to a single scalar. The optimizer knows *that* a candidate failed, but not *why*. For example, one cannot show a Bayesian optimizer a stack trace. LLM-evolution frameworks changed this by feeding execution results into LLM proposers, but when an LLM reads a compiler error, diagnoses a logic bug, and proposes a targeted fix, the process is closer to an engineer iterating on a prototype than to blind evolution.

`optany` leans into this by making diagnostic feedback a first-class part of the evaluator contract. The evaluator returns both a score and a `side_info` dictionary containing any diagnostic the evaluator can produce:

- **Text:** compiler errors, runtime exceptions, profiler summaries, natural-language critiques.
- **Structured data:** per-test-case results, sub-scores for multiple objectives, execution traces.
- **Images:** rendered SVGs, 3D model screenshots, or chart visualizations, enabling VLM proposers to *see* what they are improving.

SI is the text-optimization analogue of the gradient. Where gradients tell a numerical optimizer which direction to move, SI can tell the LLM proposer *why* a candidate failed and *how* to fix it. During a dedicated reflection step, the proposer reasons over this signal to diagnose failures and propose targeted improvements.

Prior frameworks expose feedback through framework-specific mechanisms; SI provides a uniform interface that makes it trivial to surface any diagnostic. The key design choice is that SI is *opt-in but zero-friction*: evaluators that return only a score work fine, and existing `print()` statements can be captured automatically via `capture_stdio=True`.

4.3 Pareto-Based Search

Even when optimizing a single objective, evaluating candidates across multiple examples or metrics produces richer signal than a scalar aggregate. The naive approach collapses that signal into one average score and always selects the top candidate. This stalls fast: averaging hides which aspects are strong and which are weak, and the proposer tries to improve everything at once.

`optany` does two things differently. First, it tracks scores per task (from `dataset`) or per metric (from sub-scores in SI) individually and maintains a **Pareto frontier**: any candidate that is the best at *something* survives, even if its average is suboptimal. Second, each reflection step shows the proposer a minibatch of just 2–3 examples instead of all of them, enabling focused, targeted improvements on that subset.

Over iterations, the frontier accumulates complementary strengths. Candidates that excel at different tasks are preserved and their strategies recombined. This mechanism also powers multi-task search: when optimizing across related problems, the frontier preserves candidates that excel on different tasks, and strategies discovered for one problem transfer to others (§5.8).

Candidate selection. Following GEPA [1], candidates are selected for mutation in proportion to how often they appear on the Pareto front. Let J index the objectives used to form the Pareto scores (e.g., per-example tasks, per-metric scores, or both). Each candidate Φ induces a score $s_j(\Phi)$ for every $j \in J$. Let $\mathcal{P}$ denote the set of Pareto-nondominated candidates under these objectives. For each objective $j \in J$, let $\mathcal{B}[j]$ be the set of candidates in $\mathcal{P}$ that achieve the best score on j. We sample candidates with probability proportional to $|\{j \in J : \Phi \in \mathcal{B}[j]\}|$, focusing exploration on broadly effective solutions.

Reflection and mutation. Given a selected candidate Φ and a minibatch $\mathcal{M}$ of examples, the system executes Φ on $\mathcal{M}$, collects scores and SI, and presents them to the proposer LLM in a structured reflection prompt. The proposer diagnoses failures using the SI and produces an updated artifact Φ'. If Φ' improves on the minibatch, it is fully evaluated and added to the candidate pool.

5 Experiments

We evaluate `optany` across six domains spanning all three optimization modes. For each, we describe the artifact, evaluator, SI design, and results. We then present ablation studies on multi-task search (§5.8) and SI (§5.9). Optimized solutions are presented in the Appendix E.

5.1 Coding Agent Skills (Generalization)

Setup. Skills are natural-language instructions and best practices for working with a specific codebase. The evaluator runs a coding agent on repository tasks and scores whether it resolves them; the optimized skills must generalize to unseen tasks. We optimize skills for the Bleve search library and evaluate transfer to Claude Code with both Haiku 4.5 and Sonnet 4.5.

figures/bleve_comparison_plot.png

Figure 2: Claude Code on the Bleve repository. Optimized skills boost pass rates to near-perfect while reducing resolve time by 47%. Skills transfer across models without reoptimization.

SI design. The evaluator returns task descriptions, agent traces (tool calls, code edits, errors), test outcomes, and resolution time.

Results. Optimized skills boost Haiku 4.5's pass rate from 79.3% to 98.3% and Sonnet 4.5's from 94.8% to 100%, while cutting resolution time by 47% (Figure 2). Critically, skills discovered for one model transfer effectively to another without reoptimization, demonstrating the generalization mode's ability to learn model-agnostic repository knowledge.

5.2 Cloud Scheduling Algorithms (Generalization)

Setup. We optimize two cloud infrastructure algorithms from the ADRS benchmark. **CloudCast** discovers broadcast routing strategies for multi-cloud data transfer, minimizing data egress cost. **Can't Be Late** learns scheduling policies deciding when to use cheap preemptible `SPOT` instances versus reliable `ON_DEMAND` instances to meet deadlines. Both use generalization mode with training/validation splits over infrastructure scenarios.

SI design. For CloudCast: per-partition routing decisions, edge utilizations, cost breakdowns. For Can't Be Late: spot-availability patterns, instance-usage timelines, segment counts (`SPOT` vs `ON_DEMAND` vs restarts).

Results. CloudCast achieves 40.2% cost savings over Dijkstra routing (Figure 3a), evolving from a baseline shortest-path algorithm to a provider-aware Steiner tree approach that jointly optimizes for egress cost and transfer latency. Can't Be Late achieves 7.8% cost savings (Figure 3b), evolving a simple deadline-check heuristic into an adaptive strategy with state tracking for spot-unavailability patterns, break-even switching cost analysis, and graduated decision thresholds based on slack ratio. Both results top the ADRS leaderboard, outperforming OpenEvolve, ShinkaEvolve, and expert-designed heuristics. The evolved artifacts are qualitatively different from their seeds: CloudCast discovers provider-aware routing with Steiner tree relaxation (concepts not present in the Dijkstra seed), while Can't Be Late learns to track persistent spot unavailability and compute overhead-aware switching costs (behaviors absent from the greedy seed).

cais_submission/figures/cloudcast.png

(a) CloudCast: 40.2% cost savings.

cais_submission/figures/cantbelate.png

(b) Can't Be Late: 7.8% savings.

Figure 3: Optimization trajectories for cloud scheduling. Both use generalization mode with train/val splits over infrastructure scenarios.

5.3 ARC-AGI Agent Architecture (Generalization)

Setup. Rather than optimizing a prompt, we optimize the *entire agent system*: code, sub-agent architecture, control flow, helper functions, and prompts are all treated as a single text artifact. The optimization objective is for the artifact to generalize to unseen ARC-AGI [4] puzzles.

SI design. Training/test grid examples, per-puzzle scores, internal model outputs, LLM costs, error tracebacks, and code execution results.

Results. Using Gemini 3 Flash as both the proposer and the underlying agent model, `optany` starts for a naive 10-line agent seed (one LLM call) and iteratively designs it into a 300+ line system consisting of 4 components along with fallbacks. The test accuracy improves from 32.5% to **89.5%**, a 57 percentage point gain (Figure 4). The optimized architecture implements a 4-stage pipeline: (1) rule induction via pattern analysis, (2) code generation with `exec()`-based verification, (3) iterative debugging with up to 2 fix attempts, and (4) structured fallback from code-first to direct LLM prediction. This represents a qualitative leap: the system discovers architectural patterns (verify-then-fallback, iterative refinement) that typically require manual engineering iterations.

figures/arc_agi_trajectory.png

Figure 4: ARC-AGI agent architecture evolution with Gemini 3 Flash. Validation accuracy reaches 93.5%; test accuracy improves from 32.5% to 89.5%.

5.4 AIME Prompt Optimization (Generalization)

Setup. We optimize a system prompt for GPT-4.1-mini on AIME (American Invitational Mathematics Examination) competition problems. Training uses AIME 2022–2024; testing uses AIME 2025.

Optimized Prompt for AIME

```
Solve the math problem carefully and thoroughly. Your goal is to produce a correct, well-structured solution that leads unambiguously to the requested final result. Follow these rules:
1. Restate the problem briefly in your own words.
2. Set up notation and equations cleanly before manipulating them. - Define variables explicitly. - State all constraints (e.g., integrality, ranges, geometric conditions) before using them.
3. Show clear, logically ordered reasoning. - Justify each important algebraic or geometric step. - When you split into cases, state why each case is necessary and what assumptions define it. - If you invoke a known theorem (e.g., Ptolemy, Power of a Point, similarity, Vieta), name it and show exactly how it applies in this context.
4. Handle dead ends correctly. - If you realize a line of reasoning leads to a contradiction or dead end, explicitly say so. - Then restart from the last correct point; do not guess or hand-wave.
5. Keep the reasoning focused and minimal while still being rigorous. - Avoid unnecessary numerical approximations if an exact approach is available. - Do not approximate exact values unless the problem explicitly asks for a decimal. - Prefer algebraic or structural arguments over trial-and-error or random guessing. - You may test candidate values only after deriving strong constraints that sharply limit the possibilities.
6. At the end, clearly isolate the answer: - Provide the final answer as a single number or expression on its own line. - Do not include any extra words, symbols, or explanation on that final line.
```

Figure 5: Optimized prompt for AIME

figures/aime_results.png

Figure 6: AIME prompt optimization for GPT-4.1-mini. Validation score improves from 46.67% to 57.78%; test score reaches 60.00%.

SI design. The evaluator returns each problem statement, the model's reasoning chain, extracted answer, ground truth, and a correct/incorrect flag.

Results. Prompt optimization performed through `optany` improves GPT-4.1-mini from 46.67% to **60.00%** on AIME 2025 (Figure 6), a 13.3 percentage point gain from changing only the system prompt. The optimized prompt (Figure 5) provides a structured reasoning framework with 6 explicit rules: (1) restate the problem formally, (2) set up notation before computing, (3) verify each logical step, (4) recognize and abandon dead ends, (5) stay focused on the specific question asked, and (6) isolate the final answer clearly. The seed prompt was a single generic sentence ("Solve the math problem carefully"). This result demonstrates that generalization mode can discover effective reasoning strategies from minimal seeds, consistent with the state-of-the-art prompt optimization results reported in GEPA [1]. More importantly, since these results match the performance gains reported by Agrawal et al. [1], this demonstrates that exposing a prompt optimization algorithm as an optimization backend through a general interface built for optimizing any text parameter does not hurt performance on prompt optimization.

5.5 CUDA Kernel Generation (Multi-Task Search)

Setup. We generate CUDA kernels for 31 reference PyTorch operations from KernelBench [16], evaluated on a V100 32GB GPU. The 31 problems span diverse operations: matrix multiplications, convolutions, reductions, element-wise ops, and normalization layers. Under the hood, `optany` evolves the prompt that drives kernel generation; in multi-task mode, insights discovered for one problem (e.g., how to handle memory coalescing) transfer to others automatically through the shared Pareto frontier.

SI design. The evaluator compiles the generated kernel, runs correctness tests (max absolute error vs. PyTorch reference), and benchmarks wall-clock time. SI includes: (i) NVCC compiler errors with line numbers, (ii) correctness test failures with actual vs. expected outputs, (iii) relevant CUDA documentation snippets, and (iv) speedup ratio vs. the PyTorch baseline.

figures/kernel_bench_fast_p_score.png

Figure 7: KernelBench results (GPT-5 as proposer). $\text{Fast}_p(s)$: fraction of kernels achieving speedup $\geq s$. 87% match baseline; 25% are 20%+ faster.

figures/circle_packing_annotated3_optany.png

Figure 8: Circle packing (n=26). `optany` outperforms AlphaEvolve's, ShinkaEvolve's, and OpenEvolve's solution, reaching a higher score with fewer evaluations.

Results. 87% of generated kernels match or beat the PyTorch baseline performance; 48% achieve 10%+ speedups, and 25% achieve 20%+ speedups (Figure 7). The evolved kernels employ techniques such as float4 vectorization, two-pass algorithms (compute statistics, then normalize), warp shuffle reductions, and shared memory tiling. Multi-task mode's advantages are analyzed in §5.8.

5.6 Circle Packing (Single-Task Search)

Setup. The task is to pack n=26 circles while maximizing the sum of radii within a unit square. `optany` optimizes the packing algorithm code; the evaluator executes the proposed packing code, and returns the score plus geometric diagnostics.
SI design. Circle positions, radii, constraint violations, overlap distances, boundary violations, and a rendered visualization of the packing.
Results. `optany` reaches a score of 2.63598+, outperforming AlphaEvolve's, OpenEvolve's, and ShinkaEvolve's reported solution (Figure 8). The optimized algorithm is a bilevel optimizer: an LP over radii with dual-variable gradients for L-BFGS-B center optimization, augmented by CMA-ES exploration and diverse seeding strategies.

figures/kernelbench_single_vs_batch.png

Figure 9: Single-task vs. multi-task mode on 10 selected KernelBench problems. Multi-task (blue) consistently outperforms single-task (red) at all speedup thresholds, converging faster and solving more problems.

5.7 Image Generation (Multi-Task Search)

Setup. The first task is to generate Scalable Vector Graphics (SVG) code to generate an image given a goal defined as a short description that is less than one sentence in length. We do so for three goals. We have one addition setting where we also generate code via `build123d` to generate CAD and rendering for the CAD. We present the goals in Table 3.
SI design. For the SVG, the evaluator renders the image and then queries a VLM for feedback. For each goal, we define several natural language properties which ask a VLM to rate on a scale of 0 to 100 how well the image aligns with that aspect. During each evaluator call, the VLM is asked to rate one aspect and not all at once hence the **Multi-Task Search** which allows us to explore the Pareto frontier. In the CAD setting, since we are dealing with 3D objects, we have the evaluator take 3 screenshots equidistant apart and ask the VLM to provide feedback using those images.
Results. We ask five humans (including several independent people) to examine the images created by the CAD and SVG code from a zero-shot attempt and the `optany` optimized candidates. The images are provided without identification and the humans are asked to simply pick which image they prefer. In all goals, the `optany` optimized candidates were unanimously preferred over the zero-shot attempts.

5.8 Ablation: Multi-Task vs. Single-Task Search

To gauge the effectiveness of cross-task learning, we take the 10 problems where multi-task mode performed best and re-optimize each from scratch in single-task mode. We allocate an equivalent per-problem budget to see whether a dedicated single-task run can match the multi-task result.
Figure 9 shows that multi-task mode consistently outperforms single-task search across all speedup thresholds. At the $\text{Fast}_p(1.0)$ threshold (matching baseline performance), multi-task mode solves problems faster and reaches a higher fraction solved. At $\text{Fast}_p(1.2)$ (20% speedup), the gap widens: single-task mode plateaus early, while multi-task continues improving.
The mechanism is cross-transfer via the Pareto frontier: optimization patterns discovered for one kernel (e.g., vectorized memory access, warp-level reductions) are preserved on the frontier and inform proposals for other kernels. In single-task mode, each problem must independently discover these patterns.

Goal	# Visual Aspects	Representation	Model Used in Optimization
A pelican riding a bicycle	12	SVG	`Gemini Flash 3.0`
A high-quality 3D unicorn	4	CAD	`Claude Opus 4.6`
An octopus performing on a grand pipe organ	13	SVG	`Claude Opus 4.6`
A sloth steering an excavator through a cinematic quarry at golden hour	13	SVG	`Claude Opus 4.6`

Table 3: An overview of the various target images (*goals*) from the Image Generation Experiments in §5.7. In each experiment, a VLM is used in the evaluator to provide natural langauge feedback and a score between 0 and 100 for a single visual aspect.

`figures/with_text_feedback_vs_without.png`

Figure 10: Ablation: prompt optimization with vs. without SI on the Facility Support Analysis dataset. SI accelerates convergence (left) and improves final test performance (right): 86.32 vs. 82.5.

5.9 Ablation: Side Information

To isolate the contribution of SI, we compare `optany` with and without structured diagnostic feedback (sub-scores) on prompt optimization for the Facility Support Analysis dataset. In the "with SI" condition, the evaluator returns per-aspect sub-scores alongside the aggregate score. In the "without SI" condition, only the aggregate score is returned.

Figure 10 shows two effects. First, SI accelerates convergence: the "with SI" condition reaches a validation score of 0.80 within 100 rollouts, while the score-only condition requires approximately 600 rollouts to reach the same level. Second, SI improves final performance: the test score with SI is 86.32 versus 82.5 without.

Sub-scores provide per-aspect signal: the proposer can identify that a candidate excels at aspect A but fails at aspect B, and target its revision accordingly. Without sub-scores, the proposer receives only "the overall score improved slightly," losing the diagnostic signal that enables targeted improvements.

6 Discussion

When does multi-task search help? Our experiments reveal that cross-task transfer is most beneficial when problems share underlying optimization patterns but differ in their specifics. CUDA kernel generation exemplifies this: memory coalescing, vectorized access patterns, and warp-level reductions are strategies that apply across operations but manifest differently for each kernel. Multi-task mode discovers these patterns once and transfers them, while single-task mode must rediscover them independently for each problem. We hypothesize that multi-task search will be most effective for code generation tasks where common optimization idioms exist across a family of programs.

The role of SI across domains. The SI ablation on prompt optimization demonstrates the value of structured diagnostic feedback, but the mechanism operates differently across domains. For code optimization (CUDA, circle packing), SI primarily surfaces compiler errors and runtime diagnostics that pinpoint exact failure locations. For agent architecture evolution (ARC-AGI), SI provides per-puzzle execution traces that reveal which architectural components are failing. For cloud scheduling, SI exposes the temporal structure of decisions, like when the policy chose SPOT vs. ON_DEMAND and whether that choice was optimal in retrospect. In each case, SI converts a scalar signal into actionable diagnostic information that enables the proposer to make targeted rather than random improvements.

Artifacts optimized by `optany`. A notable property of the results is the diversity of artifacts optimized through a single interface. The optimized artifacts range across structured system prompts (AIME), agent architectures (ARC-AGI) to 900+ line bilevel optimization algorithms (circle packing). The system discovers qualitatively different solution strategies: multi-stage pipelines with fallback logic (ARC-AGI), provider-aware Steiner tree algorithms (CloudCast), and break-even cost analysis with state tracking (Can't Be Late). Beyond minor refinements of the seed, they represent novel algorithmic ideas arising from the interaction between LLM reasoning and diagnostic feedback.

7 Limitations

`optany` inherits limitations from LLM-based optimization. First, the quality of proposals depends on the proposer LLM's capabilities; weaker models produce weaker candidates. Second, evaluation cost can be high when the evaluator involves expensive operations (e.g., running a full agent on many tasks). Third, the system assumes the artifact is representable as text; optimization of continuous parameters or binary artifacts requires a text-based proxy. Fourth, while multi-task search provides cross-transfer benefits on related problems, the degree of benefit depends on how related the problems are, for example, we find that circle packing does not benefit from multi-task mode. Finally, our SI ablation is conducted on a single prompt optimization task; broader ablations across domains would strengthen the finding.

8 Conclusion

`optany` demonstrates that a simple declarative interface consisting of seed artifact (or objective), evaluator and optional dataset is sufficient to express and solve diverse optimization problems as an optimization over text artifacts, matching or outperforming purpose-built tools. The key ideas are (1) three unified optimization modes under one API, (2) Side Information as a first-class evaluator contract, and (3) Pareto-based search across metrics and examples.

As LLMs become more capable, we expect the range of problems expressible as text optimization to expand further. The optany API is designed to be backend-agnostic: as new optimization strategies emerge, they can be plugged in without changing user code.

References

[1] Lakshya A Agrawal, Shangyin Tan, Dilara Soylu, Noah Ziems, Rishi Khare, Krista Opsahl-Ong, Arnav Singhvi, Herumb Shandilya, Michael J Ryan, Meng Jiang, Christopher Potts, Koushik Sen, Alexandros G. Dimakis, Ion Stoica, Dan Klein, Matei Zaharia, and Omar Khattab. 2026. GEPA: Reflective Prompt Evolution Can Outperform Reinforcement Learning. In *International Conference on Learning Representations (ICLR)*.

[2] Takuya Akiba, Shotaro Sano, Toshihiko Yanase, Takeru Ohta, and Masanori Koyama. 2019. Optuna: A Next-generation Hyperparameter Optimization Framework. arXiv:1907.10902 [cs.LG] https://arxiv.org/abs/1907.10902

[3] Angelica Chen, David Dohan, and David So. 2023. EvoPrompting: Language Models for Code-Level Neural Architecture Search. In *Advances in Neural Information Processing Systems (NeurIPS)*.

[4] François Chollet. 2019. On the Measure of Intelligence. *arXiv preprint arXiv:1911.01547* (2019).

[5] Jordan S. Ellenberg, Cristofero S. Fraser-Taliente, Thomas R. Harvey, Karan Srivastava, and Andrew V. Sutherland. 2025. Generative Modeling for Mathematical Discovery. arXiv:2503.11061 [cs.LG] https://arxiv.org/abs/2503.11061

[6] Chrisantha Fernando, Dylan Banarse, Henryk Michalewski, Simon Osindero, and Tim Rocktäschel. 2023. Promptbreeder: Self-Referential Self-Improvement Via Prompt Evolution. arXiv:2309.16797 [cs.CL] https://arxiv.org/abs/2309.16797

[7] Shengran Hu, Cong Lu, and Jeff Clune. 2024. Automated Design of Agentic Systems. In *arXiv preprint arXiv:2408.08435*.

[8] Omar Khattab, Arnav Singhvi, Paridhi Maheshwari, Zhiyuan Zhang, Keshav Santhanam, Sri Vardhamanan, Saiful Haq, Ashutosh Sharma, Thomas T. Joshi, Hanna Moazam, Heather Miller, Matei Zaharia, and Christopher Potts. 2023. DSPy: Compiling Declarative Language Model Calls into Self-Improving Pipelines. arXiv:2310.03714 [cs.CL] https://arxiv.org/abs/2310.03714

[9] Robert Tjarko Lange, Yuki Imajuku, and Edoardo Cetin. 2025. ShinkaEvolve: Towards Open-Ended And Sample-Efficient Program Evolution. arXiv:2509.19349 [cs.CL] https://arxiv.org/abs/2509.19349

[10] Joel Lehman, Jonathan Gordon, Shawn Jain, Kamal Ndousse, Cathy Yeh, and Kenneth O. Stanley. 2022. Evolution through Large Models. arXiv:2206.08896 [cs.NE] https://arxiv.org/abs/2206.08896

[11] Aman Madaan, Niket Tandon, Prakhar Gupta, Skyler Hallinan, Luyu Gao, Sarah Wiegreffe, Uri Alon, Nouha Dziri, Shrimai Prabhumoye, Yiming Yang, et al. 2023. Self-Refine: Iterative Refinement with Self-Feedback. *Advances in Neural Information Processing Systems (NeurIPS)* (2023).

[12] Michael McCourt. 2016. Optimization Test Functions. https://github.com/sigopt/evalset. https://github.com/sigopt/evalset

[13] Jean-Baptiste Mouret and Jeff Clune. 2015. Illuminating search spaces by mapping elites. arXiv:1504.04909 [cs.AI] https://arxiv.org/abs/1504.04909

[14] Alexander Novikov, Ngân Vũ, Marvin Eisenberger, Emilien Dupont, Po-Sen Huang, Adam Zsolt Wagner, Sergey Shirobokov, Borislav Kozlovskii, Francisco J. R. Ruiz, Abbas Mehrabian, M. Pawan Kumar, Abigail See, Swarat Chaudhuri, George Holland, Alex Davies, Sebastian Nowozin, Pushmeet Kohli, and Matej Balog. 2025. AlphaEvolve: A coding agent for scientific and algorithmic discovery. arXiv:2506.13131 [cs.AI] https://arxiv.org/abs/2506.13131

[15] Krista Opsahl-Ong, Michael J Ryan, Josh Purtell, David Broman, Christopher Potts, Matei Zaharia, and Omar Khattab. 2024. Optimizing Instructions and Demonstrations for Multi-Stage Language Model Programs. arXiv:2406.11695 [cs.CL] https://arxiv.org/abs/2406.11695

[16] Anne Ouyang, Simon Guo, Simran Arora, Alex L. Zhang, William Hu, Christopher Ré, and Azalia Mirhoseini. 2025. KernelBench: Can LLMs Write Efficient GPU Kernels? arXiv:2502.10517 [cs.LG] https://arxiv.org/abs/2502.10517

[17] Reid Pryzant, Dan Iter, Jerry Li, Yin Tat Lee, Chenguang Zhu, and Michael Zeng. 2023. Automatic Prompt Optimization with "Gradient Descent" and Beam Search. In *Empirical Methods in Natural Language Processing (EMNLP)*.

[18] Zhihong Shao, Peiyi Wang, Qihao Zhu, Runxin Xu, Junxiao Song, Xiao Bi, Haowei Zhang, Mingchuan Zhang, Y. K. Li, Y. Wu, and Daya Guo. 2024. DeepSeekMath: Pushing the Limits of Mathematical Reasoning in Open Language Models. arXiv:2402.03300 [cs.CL] https://arxiv.org/abs/2402.03300

[19] Asankhaya Sharma. 2025. *OpenEvolve: an open-source evolutionary coding agent*. https://github.com/algorithmicsuperintelligence/openevolve

[20] Noah Shinn, Federico Cassano, Edward Berman, Ashwin Gopinath, Karthik Narasimhan, and Shunyu Yao. 2023. Reflexion: Language Agents with Verbal Reinforcement Learning. arXiv:2303.11366 [cs.AI] https://arxiv.org/abs/2303.11366

[21] Chengrun Yang, Xuezhi Wang, Yifeng Lu, Hanxiao Liu, Quoc V. Le, Denny Zhou, and Xinyun Chen. 2024. Large Language Models as Optimizers. arXiv:2309.03409 [cs.LG] https://arxiv.org/abs/2309.03409

[22] Mert Yuksekgonul, Federico Bianchi, Joseph Boen, Sheng Liu, Zhi Huang, Carlos Guestrin, and James Zou. 2024. TextGrad: Automatic "Differentiation" via Text. arXiv:2406.07496 [cs.CL] https://arxiv.org/abs/2406.07496

[23] Jiayi Zhang, Jinyu Xiang, Zhaoyang Yu, Fengwei Teng, Xionghui Chen, Jiaqi Chen, Mingchen Zhuge, Xin Cheng, Sirui Hong, Jinlin Wang, Bingnan Zheng, Bang Liu, Yuyu Luo, and Chenglin Wu. 2025. AFlow: Automating Agentic Workflow Generation. arXiv:2410.10762 [cs.AI] https://arxiv.org/abs/2410.10762

[24] Yongchao Zhou, Andrei Ioan Muresanu, Ziwen Han, Keiran Paster, Silviu Pitis, Harris Chan, and Jimmy Ba. 2023. Large Language Models Are Human-Level Prompt Engineers. arXiv:2211.01910 [cs.LG] https://arxiv.org/abs/2211.01910

A Use of Generative AI

The authors made use of Generative AI technologies including ChatGPT, Gemini, Claude and Cursor to generate sections of this Work, including text, tables, graphs, code, etc. The experiment design and details were explicitly the authors' original ideas.

B Blackbox Mathematical Optimization

We additionally evaluate `optany` in single-task search mode on blackbox mathematical optimization, using the 56-problem EvalSet benchmark [12] against Optuna [2]. Rather than tuning parameters within a fixed algorithm, `optany` optimizes the solver code itself, discovering bespoke algorithms for each problem.
With a budget of 8,000 evaluations per problem,
`optany` ties Optuna on 40 problems, wins 7, and loses 9. On 10 selected problems where Optuna struggles with lower budgets (2,000 evaluations), `optany` finds better solutions on 7 out of 10. The mechanism: Optuna's fixed TPE-CMA-ES pipeline fails in predictable, structural ways (e.g., TPE's per-dimension sampling converges to trap basins; CMA-ES assumes smooth unimodal landscapes). `optany` tailors the solver to each problem—discovering L-BFGS-B for boundary optima and multi-start search for deceptive traps.

C Seedless Mode: 3D Unicorn

Every main experiment starts from a seed artifact. Seedless mode (`seed_candidate=None`) instead provides only a natural-language objective and lets the LLM bootstrap the first candidate. We demonstrate this on a 3D modeling task: generating a Python script (build123d + pyrender) that produces a 3D unicorn. The evaluator renders multi-view PNGs and asks a VLM to score them, passing images back as SI. Starting from no code, `optany` iteratively refines geometry, proportions, and anatomical detail, producing a recognizable 3D unicorn that improves substantially over the zero-shot baseline.

D Detailed Algorithm

Algorithm 1 `optany`: Core optimization loop

Require: Artifact Φ_0, evaluator f, dataset $\mathcal{D}$, budget B
Require: Minibatch size b, Pareto set size n
Initialize candidates $\mathcal{P} \leftarrow [\Phi_0]$
Evaluate Φ_0 on $\mathcal{D}$; record per-example scores S
while budget B not exhausted **do**
  $k \leftarrow \textsc{ParetoSelect}(\mathcal{P}, S)$ ▷ Select based on frontier
  $\mathcal{M} \leftarrow$ minibatch of size b from $\mathcal{D}$
  Execute Φ_k on $\mathcal{M}$; collect scores and SI
  $\Phi' \leftarrow \textsc{Reflect}(\Phi_k, \text{scores, SI})$ ▷ LLM proposes fix
  if Φ' improves on $\mathcal{M}$ **then**
    Evaluate Φ' on full $\mathcal{D}$
    $\mathcal{P} \leftarrow \mathcal{P} \cup \{\Phi'\}$
    Update S; prune dominated candidates
  end if
end while
return $\Phi^* \in \mathcal{P}$ maximizing average score

Algorithm 1 presents the core loop. For single-task search, the "dataset" is a singleton and per-example tracking reduces to per-metric tracking. For multi-task search, each dataset element is an independent problem. For generalization, scores on $\mathcal{D}$ guide search while a held-out `valset` measures generalization. The ParetoSelect subroutine follows GEPA [1]: it identifies non-dominated candidates and samples proportionally to their frontier frequency.

E Discovered solutions

We present excerpts of the final optimized artifacts discovered by `optany` for each domain.

E.1 Coding Agent Skills: Bleve Repository

The following is the optimized `SKILL.MD` excerpt discovered by `optany` for the Bleve search library:

Optimized Bleve Skills (excerpt)

```
4) Run tests early and iterate from failures (tests are the
bug report) - Start broad when feasible: ‘cd /testbed &&
go test ./...‘ (or project equivalent). - Narrow quickly:
- package: ‘go test ./path/to/pkg‘ - single test: ‘go
test ./path/to/pkg -run TestName -count=1‘ (add -v only
if needed) - For panics: follow the stack trace top frame
in repo code first. - For mismatches: use “expected vs
got” to locate the producing function and invariants.
...
7) Make minimal, reviewable changes and verify
continuously - Change one behavior at a time; rerun the
smallest reproducing test after each change. - Add focused
unit tests when coverage is missing; keep them in the same
package and table-driven where sensible (include short
words + accented/Unicode edge cases). - Avoid scratch
main.go files in repo root.
```

E.2 ARC-AGI Agent Architecture

The optimized agent grew from a 10-line seed to a 300+ line system implementing a 4-stage pipeline: rule induction via pattern analysis, code generation with `exec()`-based verification, iterative debugging with up to 2 fix attempts, and structured fallback from code-first to direct LLM prediction.

E.3 CloudCast Routing Algorithm

The optimized CloudCast algorithm (178 lines) discovers provider-aware Steiner tree routing with egress cost optimization, a qualitative departure from the Dijkstra seed.
We show the main `search_algorithm` function; the full artifact is available in the supplementary material.

Optimized CloudCast Algorithm (excerpt)

```
def search_algorithm(src, dsts, G, num_partitions):
    """Optimized Broadcast Routing Algorithm v3.
    Key Optimizations:
    1. Provider-Aware Weighting: biases path finding towards
       intra-provider links to minimize egress.
    2. Pareto-Frontier Candidate Selection: Explicitly keeps
       candidates that offer distinct cost/time tradeoffs.
    3. Diverse Steiner Strategies: Includes MST-like
       approximations for cost and bottleneck-widest
       paths for throughput.
    4. Robust Greedy Allocation: Accurately models bandwidth
       contention across partitions.
    """
```

cais_submission/figures/arc_agi_architecture.png

Figure 11: Architecture of the optimized ARC-AGI agent. The system discovers a 4-stage pipeline with verify-then-fallback logic, starting from a naive single-call seed.

```
    # --- Constants & Configuration ---
    EST_DATA_VOL_GB = 300.0
    EST_INSTANCE_COST_PER_HR = 10.0
    PARTITION_VOL_GB = EST_DATA_VOL_GB / max(1, num_partitions)
    # Sweep parameters for Cost vs Time tradeoff
    alphas = [0.0, 1e-5, 0.001, 0.01, 0.05, 0.1, 0.5, 2.0]
    bw_thresholds = [0.0, 0.5, 5.0, 20.0]
    strategies = ['prim', 'prim', 'furthest', 'random']
    # ... (remaining 140 lines: provider extraction, graph
    # preprocessing, Steiner tree construction, greedy
    # allocation, and Pareto-frontier selection)
```

E.4 Can't Be Late Scheduling Policy

The optimized scheduling policy (110 lines) starts from a simple deadline-check heuristic and discovers three key behaviors absent from the seed: (1) break-even switching cost analysis that avoids costly SPOT→ON_DEMAND transitions when remaining work is small, (2) persistent spot-unavailability tracking via a counter that detects when SPOT is unlikely to return, and (3) graduated decision thresholds based on slack ratio that become increasingly aggressive as the deadline approaches. We show the core `_step` method; the full artifact includes `reset()` and additional edge-case guards.

Optimized Can't Be Late Policy (excerpt)

```
from sky_spot.strategies.strategy import Strategy
from sky_spot.utils import ClusterType

class EvolveSingleRegionStrategy(Strategy):
    def __init__(self, args):
        super().__init__(args)
        self.spot_unavailable_count = 0
        self.consecutive_short_spot_windows = 0

    def _step(self, last_cluster_type, has_spot) -> ClusterType:
        remaining_task_time = self.task_duration - sum(self.
               task_done_time)
        remaining_time = self.deadline - self.env.elapsed_seconds
        slack = remaining_time - remaining_task_time - self.
               restart_overhead

        # Track persistent spot unavailability
        if not has_spot:
            self.spot_unavailable_count += 1
        else:
            self.spot_unavailable_count = 0

        # Critical deadline: must use ON_DEMAND
        if remaining_task_time + self.restart_overhead >= remaining_time -
               0.5:
            return ClusterType.ON_DEMAND

        slack_ratio = slack / max(remaining_task_time, 1e-6)

        if has_spot:
            if last_cluster_type == ClusterType.ON_DEMAND:
                # Break-even analysis: is switching to SPOT worth it?
                switch_cost = self.restart_overhead * 1.0
                savings_per_hour = 0.7 # OD(1.0) - SPOT(0.3)
                break_even = switch_cost / savings_per_hour
                if remaining_task_time < break_even * 1.5:
                    return ClusterType.ON_DEMAND
                if slack < self.restart_overhead * 3:
                    return ClusterType.ON_DEMAND
            return ClusterType.SPOT
        else:
            if last_cluster_type == ClusterType.ON_DEMAND:
                return ClusterType.ON_DEMAND
            # Graduated thresholds based on slack ratio
            if slack_ratio < 0.1:
                return ClusterType.ON_DEMAND
            if slack_ratio < 0.25 and self.spot_unavailable_count > 10:
                return ClusterType.ON_DEMAND
            if slack_ratio < 0.4 and self.spot_unavailable_count > 20:
                return ClusterType.ON_DEMAND
            return ClusterType.NONE # Wait for spot
    # ... (remaining 15 lines: reset(), _from_args(),
    # and additional small-task / tight-deadline guards)
```

E.5 CUDA Kernel: LayerNorm

We show the best individual kernel discovered for LayerNorm, which achieves a 3.32× speedup over the PyTorch baseline. The kernel employs three key techniques absent from the naive implementation: (1) `float4` vectorization that loads four values per memory transaction, cutting memory overhead by ∼4×; (2) a two-pass algorithm (compute statistics, then normalize) that lets the GPU optimize each phase independently; and (3) warp shuffle reductions (`__shfl_down_sync`) for direct register-to-register partial sum accumulation, bypassing slower shared memory paths. This kernel was discovered in multi-task mode, where optimization patterns transfer across the 31 KernelBench problems via the shared Pareto frontier.

Optimized LayerNorm CUDA Kernel (excerpt)

```
__inline__ __device__ float warp_sum(float v) {
    unsigned mask = 0xffffffffu;
    for (int offset = KB_WARP_SIZE / 2; offset > 0; offset >>= 1)
        v += __shfl_down_sync(mask, v, offset);
    return v;
}

__global__ void rowwise_stats_kernel(
    const float* __restrict__ x, float* __restrict__ mean,
    float* __restrict__ inv_std, int64_t B, int64_t M, float eps) {
    int64_t row = blockIdx.x;
    if (row >= B) return;
    const float* row_ptr = x + row * M;

    float thread_sum = 0.0f, thread_sumsq = 0.0f;
```

```
    // float4 vectorized loads when aligned
    const float4* row_v4 = reinterpret_cast<const float4*>(row_ptr);
    for (int64_t j = threadIdx.x; j < (M >> 2); j += blockDim.x) {
        float4 v = row_v4[j];
        thread_sum += (v.x + v.y + v.z + v.w);
        thread_sumsq += (v.x*v.x + v.y*v.y + v.z*v.z + v.w*v.w);
    }
    // Warp shuffle reduction + cross-warp shared memory reduce
    thread_sum = warp_sum(thread_sum);
    thread_sumsq = warp_sum(thread_sumsq);
    // ... (shared memory cross-warp reduction, mean/inv_std output)
}

__global__ void layernorm_affine_kernel(
    const float* __restrict__ x, const float* __restrict__ weight,
    const float* __restrict__ bias, const float* __restrict__ mean,
    const float* __restrict__ inv_std, float* __restrict__ y,
    int64_t B, int64_t M) {
    int64_t row = blockIdx.x;
    float m = mean[row], inv = inv_std[row];
    // Vectorized normalize + affine transform
    const float4* x_v4 = reinterpret_cast<const float4*>(x + row*M);
    float4* y_v4 = reinterpret_cast<float4*>(y + row*M);
    for (int64_t j = threadIdx.x; j < (M >> 2); j += blockDim.x) {
        float4 xv = x_v4[j], wv = w_v4[j], bv = b_v4[j];
        y_v4[j] = {((xv.x-m)*inv)*wv.x+bv.x, ((xv.y-m)*inv)*wv.y+bv.y,
                   ((xv.z-m)*inv)*wv.z+bv.z, ((xv.w-m)*inv)*wv.w+bv.w};
    }
}
// ... (remaining 80 lines: host function, input validation,
// thread/block configuration, Python module wrapper)
```

E.6 Circle Packing Algorithm

The evolved circle packing algorithm (480+ lines) is a bilevel optimizer that jointly optimizes circle centers and radii for n=26 circles in a unit square. Starting from a simple greedy packing seed, the system discovers a multi-stage architecture: (1) an LP over radii with dual-variable sensitivities that provide exact gradients for center optimization, (2) L-BFGS-B over centers using these LP-derived gradients, (3) block SLP trust-region boosts targeting the worst-performing circles, (4) CMA-ES global exploration with automatic restarts, and (5) aggressive relocation of smallest circles to edges and corners. The algorithm also employs six diverse seeding strategies (hexagonal, uniform, edge-ring, farthest-point, corner-spokes, and edge-biased hex) to avoid local optima. We show the main entry point and key optimization components.

Evolved Circle Packing Algorithm (excerpt)

```
def main(timeout, current_best_solution):
    """Bilevel L-BFGS with exact LP sensitivities +
    SLP block boosts + CMA/Evolution fallback"""
    n = 26
    # LP for optimal radii given centers; returns duals
    def solve_radii_lp(centers, need_duals=False):
        # Boundary constraints: r_i <= min(x_i, 1-x_i, y_i, 1-y_i)
        # Pairwise constraints: r_i + r_j <= ||c_i - c_j||
        # Objective: maximize sum(r)
        res = linprog(c_obj, A_ub=A_ub, b_ub=b_ub, ...)
        return r, success, {'dual': res.ineqlin.marginals}

    # Gradient from LP duals: sensitivity of sum(r) to centers
    def gradient_from_duals(centers, dual_vec):
        # Boundary: g[i,0] += (dual_left - dual_right)
        # Pairwise: g[i] += lambda_k * unit_vec(i->j)
        return g

    # Bilevel: L-BFGS-B over centers, LP over radii
    def lbfgs_bilevel(centers_init, max_iters=300):
        def f_and_g(flat):
            r, _, info = solve_radii_lp(centers, need_duals=True)
            g = gradient_from_duals(centers, info['dual'])
            return -score, -g.reshape(-1)
        minimize(f_and_g, method='L-BFGS-B', bounds=bounds)

    # Block SLP: trust-region moves on worst circles
    def block_slp_boost(centers, rounds=4, k=10, delta=0.18):
        # Jointly optimize dx, dy, r via linearized LP
        # Focus on smallest radii + highest gradient circles

    # CMA-ES global exploration over center positions
    # 6 seeding strategies: hex, uniform, edge-ring,
    # farthest-point, corner-spokes, edge-biased hex

    # Pipeline: seed evaluation -> L-BFGS bilevel ->
    # block SLP boost -> CMA-ES -> relocate worst ->
    # evolutionary exploration -> final SLP polish
    # ... (remaining 350 lines: seeding, CMA-ES, relocation,
    # evolutionary exploration, final polish)
```

Zero-shot	optany
cais_submission/svg/pelican_zero_shot.png	cais_submission/svg/pelican_optany.png
cais_submission/svg/octopus_zero_shot.png	cais_submission/svg/octopus_optany.png
cais_submission/svg/sloth_zero_shot.png	cais_submission/svg/sloth_optany.png
cais_submission/svg/unicorn_zero_shot.png	cais_submission/svg/unicorn_optany.png

Figure 12: Qualitative comparison between zero-shot generations (left) and OptimizeAnything candidates (right) across four example tasks. Optimization consistently improves many visual aspects including composition, structure, detail, and overall visual quality. In our human study, all participants preferred the `optany` generated images.